

DeepSeek Harness 白皮书 • dsh-handbook

从 0 到 1 玩转 DeepSeek Harness——官方开源 Agent 运行时的新手百科全书。★ 如果你觉得有帮助，点个 Star 支持持续更新 · 中文 · [English](#)

DeepSeek Harness 白皮书

从 0 到 1 玩转官方开源 Agent 运行时 · 新手百科全书

dsh-handbook

7 章 · 中英双语 · PDF
benchmark · demo · 插件实战

dsh-handbook 白皮书 章节 7 PDF 1.25MB license CC-BY-NC-SA-4.0 dsh 0.1.0-rc.6

这是什么

DeepSeek Harness (dsh) 是 DeepSeek 官方 2026-08-13 开源的 Agent 运行时——一个"一切皆插件" (everything is a plugin) 的框架。但官方文档以架构说明为主，**缺少一条从零上手的路径**。

这本白皮书补上这条路：从"什么是 Agent 运行时"讲起，到安装、使用、开发插件、性能调优——每一章都有可复制、可运行的命令，全部在本机实测验证。**目标是：任何一个开发者，跟着这本书都能从 0 到 1 用起来、写起来。**

快速体验 (30 秒)

```
# 1. 安装 (需要 Node.js ≥ 22)
npx -y @deepseek-ai/dsh web

# 2. 浏览器打开 http://127.0.0.1:3080, 开始对话
# 3. 或跑一次性任务 (适合脚本/CI)
dsh --profile headless "你好，请用一句话介绍自己"
```

想系统学？看 [🐾 学习路径 \(3 天计划\)](#)；想先跑？[第 2 章：五分钟快速上手](#)

目录 (从 0 到 1)

#	章节	你将学会	状态
	学习路径 (3 天计划)	从 0 到 1：每天目标 + 验收标准 + 学习原则	✅
1	认识 DeepSeek Harness · EN	它是什么、为什么值得学、与主流 Agent 全面对比、FAQ	✅ 双语
2	五分钟快速上手 · EN	安装、web/headless 双模式、模型与推理档位、排障	✅
3	profile 与插件系统	可定制骨架、插件挂载、host/client 双半、扩展点、真实坑	✅
4	插件开发实战	从零写第一个插件 (完整代码 + 测试 + 实机验证)	✅
5	实战案例	三个真实开源 PR 的完整闭环	✅
6	进阶与性能调优	推理档位策略、耗时分析、踩坑清单	✅

7	生态与资源	官方入口、参与路径、阅读建议	✓
8	工具与上下文系统	60+ 能力包地图、内置工具、上下文注入、compaction、安全模型	✓
9	MCP、子代理与工作流	外部工具接入、并行子代理、多步编排、Agent 系统蓝图	✓
附	术语表与命令速查 · Benchmark	30+ 术语、命令速查、3 Agent 实测	✓

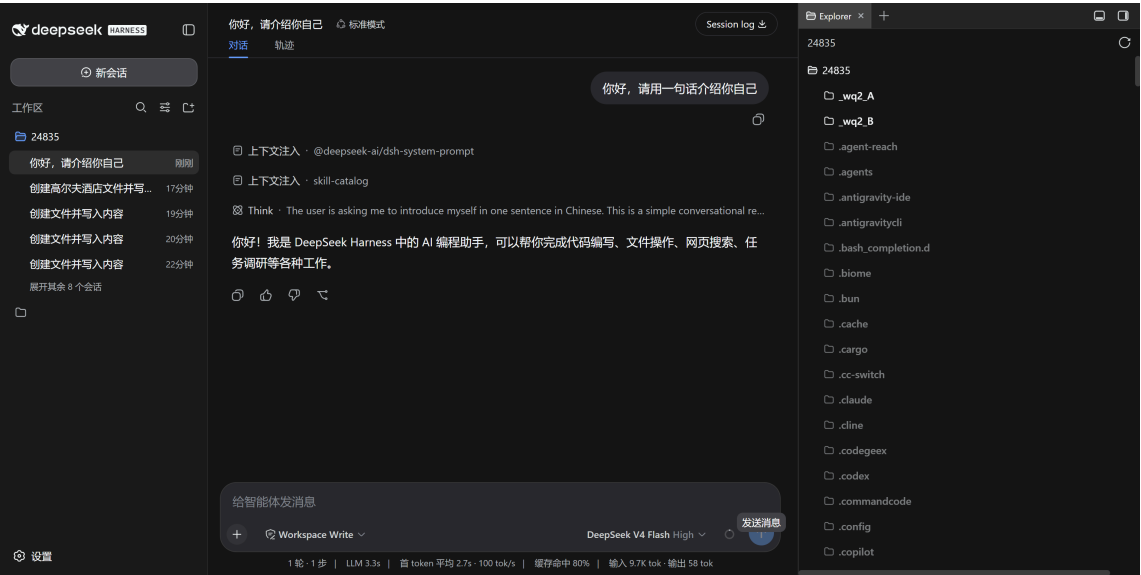
内容精华速览（点开即看，不止链接）

- ▶ 📘 第 1 章：认识 DeepSeek Harness —— 三个直觉 + 能力矩阵
- ▶ ⚡ 第 2 章：五分钟快速上手 —— 30 秒跑起来
- ▶ 🧩 第 3 章：profile 与插件系统 —— 可定制骨架
- ▶ ✂️ 第 4 章：插件开发实战 —— 完整可运行代码
- ▶ 📦 第 5 章：实战案例 —— 三个真实开源 PR 的完整闭环
- ▶ 🚀 第 6 章：进阶与性能调优 —— 时间花在哪
- ▶ 🛠️ 第 8 章：工具与上下文系统 —— 能力引擎
- ▶ 🔗 第 9 章：MCP、子代理与工作流 —— Agent 系统化
- ▶ 📖 附录：术语表 + 命令速查
- ▶ 🌐 第 7 章：生态与资源 —— 加入 dsh 生态的地图

演示（Demo）—— 直接看效果

① Web UI 对话（dsh web）

```
dsh web    # → http://127.0.0.1:3080
```

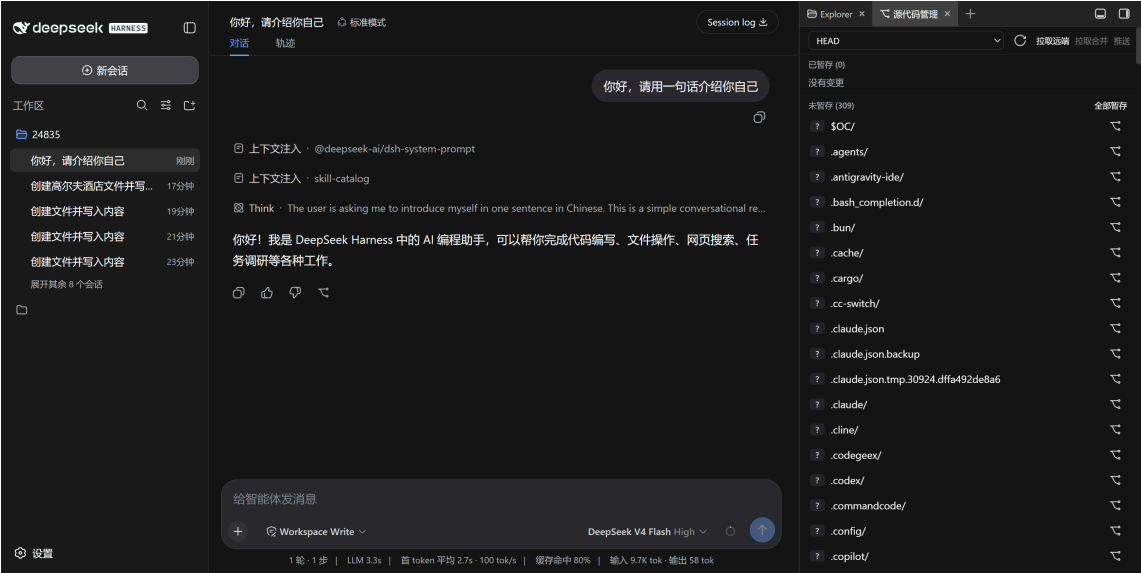


② Headless CLI（一次性任务，适合脚本/CI）

```
dsh --profile headless "你好，请用一句话介绍你自己"
# → 你好！我是 DeepSeek 驱动的 AI 编程助手，可以帮你写代码、调试问题、
```

处理文件、搜索资料，以及完成各种开发和办公任务。

③ 插件生态（Git 面板， dsh-better-sidebar ）



完整图文演示见 [30 秒看懂 dsh](#)。

DSH vs 主流 Agent（能力矩阵）

维度	dsh	Claude Code	OpenAI Codex	OpenCode	Gemini CLI	Kimi CLI
开源	✔ MIT	✘	✘	✔ MIT	✘	✘
模型绑定	模型无关	Claude 系	GPT 系	任意	Gemini 系	Kimi 系
插件体系	官方级：一切皆插件，60+ 官方包	配置/钩子	配置	配置	无	无
自定义界面	✔ (client 半)	✘	✘	部分	✘	✘
自动化/CI	✔ headless	✔	✔	✔	✔	✔
TUI	插件可做	✔ 内置	✔ 内置	✔ 内置	✔	✔
生态阶段	零日（2026-08-13）	成熟	成熟	成熟	成熟	早期
适合谁	深度定制+生态	开箱即用	开箱即用	OpenCode 用户	Google	Kimi

实测案例、同模型多 Agent 对比数据见 [第 1 章](#) 与 [benchmark](#) 章节。

同模型 × 不同 Agent 实测 (2026-08-13)

模型统一 `deepseek-v4-flash` (同一网关、同一 key) , 只对比 Agent 工程层。3 任务全部正确完成, 差异在效率:

Agent	总耗时	正确率
omp	36s	27/27
dsh	85s	27/27
opencode	114s	27/27

3 轮采样中位数, 27/27 全对。完整方法/解读见 [Benchmark 附录](#)。

白皮书 PDF

- 中文完整版: [DeepSeek-Harness-白皮书.pdf](#) (7 章合订, 1.25MB)
- 英文版 PDF 随英文章节完成度更新

为什么值得读 (而不是只看官方文档)

官方文档	本白皮书
架构视角 (AGENTS.md / architecture.md)	新手视角: 一条从 0 到 1 的路径
零散示例	每章可运行, 命令全部实测
无中文教程	中文优先, 英文同步
无生态实操	真实插件/PR 拆解 (含踩坑与安全约束)

与生态联动

本白皮书的方法论来自真实开源实践:

- [dsh-tool-turbo](#) —— 工具调用提速插件 (第 4/6 章源码)
- [DSH-better-sidebar](#) —— 社区侧边栏插件 (第 5 章案例)

贡献与反馈

- 章节/命令失效? rc 版本迭代所致, 欢迎 issue 指正
- 想参与? 见 [第 7 章: 生态与资源](#)

学习路径: 从 0 到 1 的 3 天计划

承诺兑现: 任何开发者, 3 天从零到能开发插件。本路径把 9 章拆成 3 天, 每天有明确目标与验收标准。

Day 1: 理解 + 跑起来 (第 1-2 章)

目标: 理解 dsh 是什么, 能独立启动、对话、配置。

时间	任务	验收
30 分钟	读第 1 章 (三个直觉 + 能力矩阵)	能向别人解释"dsh 和 Claude Code 的区别"

30 分钟	第 2 章动手: dsh web 对话 + headless	能启动 UI 完成一次对话; 能跑一条 headless 命令
30 分钟	配置模型/推理档位 + 排障速查	能改 settings.yaml; 能解决端口占用

验收: `dsh --profile headless “你好”` 有回复。

Day 2: 理解骨架 + 第一个插件 (第 3-4 章)

目标: 理解 profile/插件机制, 写出第一个能跑的插件。

时间	任务	验收
1 小时	第 3 章 (profile、挂载、host/client、扩展点、坑)	能挂载社区插件 (Git 面板)
1.5 小时	第 4 章抄写 tool-turbo: 纯函数 + agent/request 注入	插件加载无错, 日志出现决策输出

验收: 自己的插件在 dsh 里生效 (日志证明注入发生)。

Day 3: 实战 + 调优 (第 5-6 章)

目标: 能评估插件质量、调优性能、理解生态玩法。

时间	任务	验收
1 小时	第 5 章三个真实 PR (安全红线、沙箱约束、测试方法)	能说出“写一个 PR 的完整闭环”
1 小时	第 6 章性能模型 (思考占 90%) + 档位策略 + 坑	能给一个任务选档位、预估瓶颈
30 分钟	第 8-9 章扫读 (工具/上下文、MCP/子代理/工作流) + 术语表	知道 dsh 的能力全貌与扩展方向

验收: 能给 dsh 生态贡献一个小改进或写一篇使用心得。

之后: 按需深入

想做什么	去读
接外部工具	第 9 章 MCP + 官方 mcp 包文档
并行任务	第 9 章子代理
确定性流程	第 9 章工作流 + 官方 examples
生产部署	官方架构文档 + 安全模型 (sandbox/权限)
性能极致	第 6 章 + dsh-tool-turbo 源码
生态变现	第 7 章 + dsh-plugin topic

学习原则 (白皮书全程)

- 1. 动手 > 阅读: 每章命令都跑一遍, 不看会后悔
 - 2. 先复制后理解: 第 4 章完整代码先跑通, 再改参数理解机制
 - 3. 单测 + 实机双证据: 开发插件时两个都要过
 - 4. 善用 `--dump-config`: 疑惑时看合成配置, 比猜快
-

第 1 章：认识 DeepSeek Harness

本章目标：从一个完全零基础的角度，建立对 DeepSeek Harness (dsh) 的完整认知——它是什么、为什么值得学、和主流 Agent 有什么区别、什么时候用它。读完本章，你不需要任何前置知识，就能理解 dsh 在整个 AI 开发工具版图中的位置。

1.1 先建立三个直觉

在讲技术之前，先用三个类比建立直觉：

- ① dsh 是什么？——它是"Agent 的乐高底座"。想象乐高：官方提供底板和标准积木（运行时 + 核心插件），你可以自由拼装（加插件、换界面、改行为）。对比之下，Claude Code 更像"一辆整车"——很好开，但你想改装发动机得找官方。
- ② 为什么需要"harness"这个词？——它是"套在模型外面的工程层"。一个模型（DeepSeek V4）本身只会"回复文字"。要让它在你的代码仓库里干活（读文件、跑命令、改代码、多轮循环），外面需要一层工程：会话管理、工具调用、上下文控制、错误恢复。这层工程就叫 harness。dsh 是 DeepSeek 官方把这层工程开源出来的产品。
- ③ 为什么 2026 年才开源？——因为 Agent 进入"可编程时代"。2025 年是"模型能力竞赛"（谁能生成更好的代码）；2026 年进入"Agent 工程竞赛"（谁能更好地组织模型干活）。DeepSeek 开源 harness 的战略意图：把"如何组织 Agent"这件事变成开放生态——像当年 Android 开源改变手机生态一样。

1.2 官方定义与核心事实

一句话定义：DeepSeek Harness (dsh) 是 DeepSeek 官方开源的 Agent 运行时，采用"一切皆插件" (everything is a plugin) 架构，基于 Cordis 插件容器构建。

事实	内容
开源时间	2026-08-13
协议	MIT (可商用、可改)
语言	TypeScript (Node.js ≥ 22)
版本线	0.1.0-rc.x (当前 rc.6, 迭代快, 官方明示"将有破坏性变更")
底层	Cordis (可组合插件容器)
内置形态	web (Web UI) + headless (一次性 CLI)
官方定位	官方 README 原话: "everything is a plugin"

1.3 架构是怎么"一切皆插件"的

3.1 分层视图

```
| 你的 profile (可启动形态) |
| = bundle 栈 + 你的补丁层 |
|   • dsh web (Web UI 形态) |
|   • dsh headless (CLI 形态) |
|   • 你的自定义 profile (TUI/桌面/机器人...) |
```

能力层（每个能力 = 一个插件）	
• llm: 模型接入（DeepSeek V4 系 + 推理档位）	
• tools: 工具（写文件/终端/搜索/技能…）	
• session: 会话持久化	
• client: 界面（web 浏览器半 / 终端半）	
• settings: 用户配置	
…（60+ 官方包）	

Cordis 插件容器：加载、依赖注入、事件、生命周期	

3.2 三个必须懂的概念

① profile（可启动形态） 一个 profile = \$DSH_HOME/profiles/<name>/ 目录，包含：

- package.json：插件依赖 + 清单（dsh.profile.bundles 指定 bundle 顺序）
- cordis.patch.yml：你的补丁层（挂载/覆盖插件） 启动时按顺序合成：内置 bundle → profile patch → 全局 patch → --patch 覆盖。

② host 半 / client 半（一个插件，两副面孔）

半边	跑在哪	干什么
host 半	Node 进程	工具、服务、事件、文件系统——apply(ctx) 注册
client 半	浏览器（web profile）	UI、交互——package.json 的 dsh.client 声明

一个 npm 包可同时携带两半（exports["."] + exports["./client"]）。

③ 扩展点（extension point） 官方原则："Plugins, not loop changes"——改行为优先用官方钩子，不要 fork 核心。常用扩展点（后续章节逐一实战）：

- agent/request waterfall：每次模型请求前改配置（工具调用提速插件的挂点）
- conversationEvents.register：订阅/注入对话事件
- ctx.slots.inject：在界面槽位注入 UI
- settings 服务：注册用户可配置项

1.4 DSH 与主流 Agent 的全面对比

4.1 能力矩阵

维度	dsh	Claude Code	OpenAI Codex	OpenCode	Gemini CLI	Kimi CLI
开源	✔ MIT	✗ 闭源	✗ 闭源	✔ MIT	✗ 闭源	✗ 闭源
模型绑定	模型无关（官方适配 DeepSeek）	Claude 系	GPT 系	任意	Gemini 系	Kimi 系
官方运行时	✔（web + headless + 插件生态）	产品即运行时	产品即运行时	客户端（无官方后端）	产品即运行时	产品即运行时
插件体系	官方级：一切皆插件，60+ 官方包	配置/钩子为主	配置为主	配置为主	无	无

自定义界面	✅ (client 半 = 自由 UI)	❌	❌	部分 (TUI 固定)	❌	❌
自动化/CI	✅ headless profile	✅	✅	✅	✅	✅
TUI	插件可做 (官方未内置)	✅ 内置	✅ 内置	✅ 内置	✅	✅
生态阶段	零日起步 (2026-08-13)	成熟	成熟	成熟	成熟	早期
适合谁	想深度定制 + 玩生态的开发者	开箱即用	开箱即用	熟悉 OpenCode 用户	Google 生态	Kimi 生态

4.2 案例对比：同一个任务，不同的打开方式

任务：让 Agent 在仓库里"找到所有调用某个函数的地方，并统一改一个参数"。

Agent	你会怎么做	体验
dsh (web)	dsh web → 输入指令 → 模型用 Grep/Read/Edit 工具完成	Web UI + 右侧插件侧边栏 (可加 Git 面板)
dsh (headless)	dsh --profile headless "任务" → 打印结果退出	可进 CI：非零退出码即失败
Claude Code	claude → 输入指令 → 模型完成	终端 TUI，开箱即用
Codex	codex → 输入指令	终端 TUI
OpenCode	opencode → 输入指令	终端 TUI，可配任意模型
Kimi CLI	kimi → 输入指令	终端 TUI

差异点在哪：同样一句话，dsh 让你多了一个选择维度——界面和工具链都可以换。比如把 Git 面板、工具调用提速插件挂进去，同一个模型干活的效率就不一样了。其他 Agent 的界面/工具链是官方定的。

4.3 案例对比：真实 workflow（我们的实测）

以下是我们真实开发 dsh 生态时的对比观察 (2026-08-13)：

场景	dsh 实测	备注
简单文件创建	冷启动 ~110s (首轮含上下文注入) → 热缓存 ~1s	思考档位是主要变量
50 步工具链任务	LLM 耗时 10m+, 工具调用 9m+	每步思考累计——提速插件价值在此
插件开发	从零到可运行插件：1 天 (含测试+实机验证)	扩展点清晰 (agent/request waterfall)
与 Claude Code 同任务	dsh 配 V4-Flash 成本约为 Claude 的 1/10~1/30	价格维度 dsh 生态显著占优

数据来源：本白皮书配套开发记录 (dsh-tool-turbo 实测日志 + 官方成绩单对比)。

1.5 什么时候用 dsh（选型决策）

✅ 推荐入场：

- 你要做模型无关、界面可选、行为可改的 Agent 底座
- 你想成为 dsh 生态早期贡献者（先发优势，官方点名鼓励）
- 你要在服务器/CI 跑 Agent（headless profile）
- 你对成本敏感（DeepSeek 模型 + 开源生态）

⏸ 暂时观望：

- 只要“开箱即用的编码助手”——Claude Code 等更成熟
- 不能接受 rc 阶段的破坏性变更——等 0.1.0 正式版
- 重度依赖某模型独有能力（如 Claude artifacts）——模型绑定场景

1.6 常见问题（FAQ）

Q1: dsh 是模型吗？不是。dsh 是运行时/框架，模型通过 `llm` 插件接入（官方适配 DeepSeek V4 系，理论上可接其他 OpenAI 兼容模型）。

Q2: dsh 和 OpenCode 什么关系？都是开源 Agent 客户端，但定位不同：OpenCode 是“客户端 + 配置”，dsh 是“运行时 + 官方插件生态”（有官方后端 bundle 与 60+ 官方包）。dsh 更底层、可定制面更大。

Q3: 没写过 TypeScript 能玩吗？能。使用（第 2 章）不需要编程；写插件（第 4 章）需要基础 TS，但教程给完整代码。

Q4: dsh 稳定吗？当前 rc 阶段（0.1.0-rc.6），迭代快、有破坏性变更。生产核心依赖建议等正式版；玩生态现在是时机。

Q5: 为什么现在学 dsh 值得？生态零日 + 官方点名鼓励社区 + 中文教程空白——每个早期生态都有“第一个吃螃蟹的人”的红利，现在是入场窗口。

下一章：第 2 章：五分钟快速上手 —— 装起来，跑起来。

第 2 章：五分钟快速上手

本章目标：跟着做，跑起来。每一条命令都给出预期输出与常见错误解法。建议打开终端边看边做。

2.1 准备工作（30 秒检查）

需要	检查命令	通过标准
Node.js ≥ 22	<code>node --version</code>	v22.x 或更高（推荐 24）
npm（随 Node 附带）	<code>npm --version</code>	有版本号即可
网络	能访问 npm registry	能装包
（可选）DeepSeek API Key	https://platform.deepseek.com	用于真实对话

没有 API Key 也能启动 dsh（界面能开），但对话需要 Key。本白皮书示例假设已配置。

2.2 安装（两种方式）

方式一：直接运行（推荐新手）

```
npx -y @deepseek-ai/dsh --version
```

首次运行会下载 dsh（包体较大，含 40+ 插件模块，约 1-3 分钟）。看到版本号即成功：

```
0.1.0-rc.6
```

方式二：全局安装（推荐频繁使用）

```
npm install -g @deepseek-ai/dsh
dsh --version
```

2.3 模式一：Web UI（dsh web）

启动

```
dsh web
```

预期输出：

```
dsh web: http://127.0.0.1:3080
```

浏览器打开 <http://127.0.0.1:3080>。

界面认识（对照截图）

 dsh Web UI 对话

区域	内容
左栏	会话列表 / 工作区切换 / 新建会话
中栏	对话区：输入框、模型选择（DeepSeek V4 Flash）、推理等级（High）
右侧/底部	插件侧边栏（默认空；安装社区插件后出现）
右上	Session log（会话日志） / 轨迹（工具调用轨迹）

第一次对话

- 点「新建会话」
- 输入框输入：你好，请用一句话介绍你自己
- 回车发送

预期回复类似：

你好！我是 DeepSeek 驱动的 AI 编程助手，可以帮你写代码、调试问题、处理文件、搜索资料，以及完成各种开发和办公任务。

模型与推理档位

点输入框旁的「选择模型」：

模型	定位
----	----

deepseek-v4-flash (默认)	性价比：快、便宜，日常够用
deepseek-v4-pro	旗舰：更强，更贵更慢

推理等级（思考模式三档，2026-08-13 起支持）：

档位	速度	质量	建议场景
low	最快	够用	简单/确定性任务、批量、工具链廉价轮次
high (默认)	中等	好	日常 Agent 任务
max	最慢	最强	复杂推理、长链规划

💡 性能关键认知：模型在每次工具调用前都会重新思考。实测一个“创建文件”任务，思考占 ~90% 墙钟时间；50 步工具链任务思考累计可达数分钟到十几分钟。调低推理档位是性价比最高的提速手段（见第 6 章 + dsh-tool-turbo 插件）。

2.4 模式二：Headless（一次性任务，适合脚本/CI）

```
dsh --profile headless "你好，请用一句话介绍你自己"
```

预期输出（打印结果后进程退出）：

你好！我是 DeepSeek 驱动的 AI 编程助手，可以帮你写代码、调试问题、处理文件、搜索资料，以及完成各种开发和办公任务。

Headless 的核心价值：

- 自动化：可进 CI、服务器、cron
- 脚本友好：非零退出码 = 失败；输出可管道处理
- 会话隔离：每次调用一个新鲜会话（--resume 可恢复，见 dsh --profile headless --help）

实战：写个脚本每天跑一次“生成日报”：

```
dsh --profile headless "读取工作区今天的 git log，生成一份中文日报摘要" > daily-report.md
echo "exit=$?"
```

2.5 你的第一个插件：给 web 加个 Git 面板

dsh 的侧边栏默认是空的——安装社区插件 dsh-better-sidebar 体验“一切皆插件”（详细原理见第 3 章，这里先跑通）：

```
# 1. 找到你的 web profile
#   Windows: %USERPROFILE%\dsh\profiles\web
#   macOS/Linux: ~/.dsh/profiles/web

# 2. 在 package.json 的 dependencies 加一行（link: 指向插件源码）
#   "dsh-better-sidebar": "link:C:\\path\\to\\DSH-better-sidebar"

# 3. 在 cordis.patch.yml 加挂载行
#   - insert:
```

```
#           - id: better-sidebar
#           name: dsh-better-sidebar

# 4. 安装并重启
cd ~/.dsh/profiles/web && pnpm install
dsh web
```

重启后，右侧出现文件管理 / 终端 / Git 面板 / 浏览器等标签：

 dsh Git 面板 (better-sidebar 插件)

图中「拉取远端 / 拉取合并 / 推送」按钮是社区 PR 实现的（见第 5 章案例）——这就是“插件生态”的运转方式。

2.6 配置与目录速查

首次运行后生成的目录：

```
~/.dsh/
├── settings.yaml          # 全局设置（模型、推理档位）
├── profiles/             # profile 目录
│   └── web/
│       ├── package.json  # 插件依赖 + 清单
│       └── cordis.patch.yml # 补丁层（挂载插件）
├── sessions/             # 会话数据
└── storages/              # 持久化存储
```

settings.yaml 示例：

```
agent-default-model:
  model: deepseek-v4-flash
  reasoningEffort: high
```

2.7 命令速查

命令	用途
dsh web	启动 Web UI (=dsh --profile web)
dsh --profile headless “任务”	一次性任务，打印结果退出
dsh plugin --profile <name> add <pkg>	给 profile 安装插件
dsh --dump-config	打印合成配置树
dsh --profile tui	TUI 模式（需先安装 tui 插件，官方未内置）
dsh --version	版本

2.8 排障速查

现象	原因与解法
----	-------

dsh: profile "tui" does not exist	tui profile 需插件创建 (dsh plugin --profile tui add <pkg>)
npx 极慢	首次下载包体大; npm i -g 后更快
浏览器打不开 3080	端口被占: netstat -ano findstr 3080 → kill PID
模型无响应	检查 ~/.dsh/settings.yaml 模型配置 + API Key
插件装不上 (404)	rc.1 依赖断裂: 确认依赖用 ^0.1.0-rc.6 线 (第 3 章常见坑 #1)
升级后行为变了	rc 阶段破坏性变更正常, 看官方 changelog

下一章: [第 3 章: profile 与插件系统](#) —— 理解可定制骨架。

第 3 章: profile 与插件系统

本章目标: 理解 dsh 的可定制骨架——profile 怎么组织、插件怎么挂载、host/client 双半是什么。这是从“用户”变成“开发者”的分水岭。

3.1 profile: 一个可启动的配置栈

dsh 用 profile 表示“一种可启动的形态”。官方内置两个，其余用插件创建：

profile	用途	命令
web	Web UI (对话 + 侧边栏 + 工具)	dsh web
headless	一次性 CLI 任务	dsh --profile headless “任务”
tui (需插件)	终端 UI	dsh --profile tui (未内置, 需安装插件)

一个 profile 目录长这样 (~/.dsh/profiles/<name>/)：

```
profiles/web/
├── package.json      # 插件依赖 + dsh.profile 清单 (bundles 顺序)
├── cordis.patch.yml  # 你的补丁层: 挂载/覆盖插件的声明
├── cordis.yml        # (生成的) 合成配置
├── pnpm-workspace.yaml
└── node_modules/
```

加载顺序 (官方文档原文)： 内置 bundle (dsh-base → dsh-web-app) → **profile 的** cordis.patch.yml → **用户级** ~/.dsh/cordis.patch.yml → --patch 覆盖层。

3.2 挂载一个插件: 两处改动

以挂载社区插件 dsh-better-sidebar 为例 (真实操作)：

① package.json 加依赖 (link: 指向本地源码, 或用 npm 包名)：

```
{
  "dependencies": {
    "dsh-better-sidebar": "link:C:\\path\\to\\DSH-better-sidebar"
```

```
}  
}
```

② cordis.patch.yml 加挂载行：

```
- insert:  
  - id: better-sidebar  
    name: dsh-better-sidebar
```

③ 安装并重启：

```
cd ~/.dsh/profiles/web && pnpm install  
dsh web # 重启后生效
```

💡 插件默认是按会话隔离的 (*better-sidebar* 的布局/标签按会话持久化) ——挂载是 *profile* 级，但状态是会话级。

3.3 host 半与 client 半：一个包，两副面孔

插件可以同时携带两个运行半：

半边	运行位置	职责	示例
host 半	Node 进程	工具、服务、事件、文件系统、进程	<code>apply(ctx)</code> 注册工具/服务
client 半	浏览器 (web profile)	UI、交互、DOM	<code>package.json</code> 的 <code>dsh.client</code> 声明 + <code>src/client/</code>

`package.json` 里如何声明 client 半：

```
{  
  "dsh": {  
    "client": {  
      "inject": ["@deepseek-ai/dsh-client-runtime", "@deepseek-ai/dsh-client-locale"],  
      "platform": "web"  
    }  
  },  
  "exports": {  
    ".": { "types": "./lib/types/index.d.ts", "default": "./lib/index.js" },  
    "./client": { "types": "./lib/types/client/index.d.ts", "default":  
      "./lib/client.js" }  
  }  
}
```

- **host 半**： `exports["."]` → `apply(ctx)` (**cordis 插件主体**)
- **client 半**： `exports["./client"]` → 浏览器侧的 `apply(ctx)`
- **inject**：声明需要的服务 (**cordis 依赖注入**)

3.4 扩展点：改行为优先找钩子，别 fork 核心

官方原则 (CONTRIBUTING/AGENTS.md 明示) : *****Plugins, not loop changes: new behavior goes on documented extension points*****。新手最常见的错误是改核心 loop——正确做法是用扩展点。

已确认的常用扩展点 (后续章节逐一实战) :

扩展点	位置	用途
agent/request waterfall	agent-loop	每次模型请求前改配置 (provider/model/reasoningEffort/tools) —— tool-turbo 用这个
agent/request-error	agent-loop	请求失败时干预 (官方 compaction 插件用这个做上下文溢出恢复)
conversationEvents.register	client runtime	订阅/注入对话事件 (tool/call、turn/start 等)
ctx.slots.inject	client ui-slots	在界面槽位注入 UI (如 turnTail 显示产物文件行)
settings 服务	dsh-settings	注册用户可配置的命名空间 (设置页自动渲染)
ctx.provide / ctx.get	cordis	跨插件提供服务

3.5 常见坑 (真实踩过)

坑	现象	解决
rc.1 依赖断裂	pnpm install 报 @deepseek-ai/dsh-type-meta@0.0.1-rc.1 404	官方 rc.1 时代多个包从未发布; 升级到 ^0.1.0-rc.6 线
插件缺 main	dsh: No "exports" main defined	host 插件 package.json 要暴露 . 入口 ("main": "src/index.ts" 可被 tsx 直接加载)
事件 handler 忘了 await next()	请求丢失 provider/model 报错	agent/request 的 next() 返回 Promise, 必须 await 后 spread
类型报 'agent/request' is not assignable to keyof Events	npm 类型未 re-export 官方类型增强	用宽松签名 (ctx.on as unknown as ...) 在边界转换
client 包产物依赖 window.__ModuleLoader__	jsdom 无法直接跑 client 测试	组件级测试需 dsh web 运行时 (官方 CI 跑)

下一章: [第 4 章: 插件开发实战](#) (规划中) —— 从零写第一个 host 插件。

第 4 章: 插件开发实战

本章目标: 从零写一个真实可用的 host 插件——通过 agent/request 扩展点自动调节推理档位。这是社区项目 dsh-tool-turbo 的完整拆解, 所有代码可运行、可测试。

4.1 我们要做什么

问题：dsh 在每次工具调用前模型都会重新思考（`reasoning_effort`）。一个 50 步工具链任务，“思考”占 90%+ 墙钟时间。

方案：一个 host 插件，监听 `agent/request` waterfall，根据当前步骤最近的工具调用，把简单轮次的 `reasoning_effort` 从 high 降到 low。

4.2 项目骨架

```
dsh-tool-turbo/
├── package.json          # host 插件声明
├── tsconfig.json
├── src/
│   ├── effort-decision.ts # 纯函数：决策逻辑（零依赖，可单测）
│   └── index.ts           # apply(ctx)：接入扩展点
└── tests/
    └── effort-decision.spec.ts
```

package.json 关键字段：

```
{
  "name": "dsh-tool-turbo",
  "type": "module",
  "main": "src/index.ts",
  "exports": {
    ".": { "types": "./src/index.ts", "default": "./src/index.ts" }
  },
  "peerDependencies": {
    "@deepseek-ai/cordis": "^4.0.1",
    "@deepseek-ai/dsh-agent": "^0.1.0-rc.6"
  }
}
```

⚠ 依赖版本务必用 `^0.1.0-rc.6` 线——`rc.1` 线的 npm 依赖链是断的（见第 3 章常见坑）。

4.3 纯函数：决策逻辑（零依赖，可单测）

src/effort-decision.ts：

```
export type EffortId = 'low' | 'high' | 'max'

export interface ToolCallSample {
  name: string // 工具名，如 'write'、'read'、'bash'
  argsSize: number // 参数大小（字符数）
}

export interface EffortDecisionInput {
  recentCalls: readonly ToolCallSample[]
  selected: EffortId // 用户基线档
  allowDowngrade: boolean
```



```

    allowUpgrade: boolean
  }

const SIMPLE_TOOL_RE =
/^ (fs|bash|terminal|read|write|grep|glob|edit|ls|cat|rm|cp|touch|mkdir|pwd)/i
const HEAVY_ARGS = 800

export function decideEffort(input: EffortDecisionInput): EffortId {
  const { recentCalls, selected, allowDowngrade, allowUpgrade } = input
  if (recentCalls.length === 0) return selected // 全新提示：保持基线

  const ratio = recentCalls.filter(c =>
    SIMPLE_TOOL_RE.test(c.name) && c.argsSize < HEAVY_ARGS,
  ).length / recentCalls.length
  const heaviest = recentCalls.reduce((m, c) => Math.max(m, c.argsSize), 0)

  if (ratio >= 0.75 && allowDowngrade) return 'low'
  if (heaviest >= HEAVY_ARGS * 4 && allowUpgrade) return 'max'
  if (ratio < 0.75) return allowUpgrade ? 'high' : selected
  return selected
}

```

为什么拆成纯函数：决策逻辑与 dsh 运行时解耦——单元测试零依赖、毫秒级、覆盖所有分支，实机只需要验证“注入是否真的发生”。

4.4 插件主体：接入 agent/request waterfall

src/index.ts :

```

import type { Context } from '@deepseek-ai/cordis'
import { decideEffort, type ToolCallSample } from './effort-decision.ts'

export interface ToolTurboConfig {
  enabled: boolean
  allowDowngrade: boolean
  allowUpgrade: boolean
  baseline: 'low' | 'high' | 'max'
}

export const DEFAULT_CONFIG: ToolTurboConfig = {
  enabled: true, allowDowngrade: true, allowUpgrade: false, baseline: 'high',
}

const WINDOW = 8

function recentToolCalls(agent: unknown): ToolCallSample[] {
  const events = (agent as { session?: { events?: readonly unknown[] } })
    .session?.events ?? []
  const out: ToolCallSample[] = []
  for (let i = events.length - 1; i >= 0 && out.length < WINDOW; i--) {
    const e = events[i] as { type?: string; data?: { name?: string; arguments?:

```

```

unknown } } | undefined
    if (e?.type !== 'tool/call') continue
    out.push({
      name: e.data?.name ?? 'tool',
      argsSize: typeof e.data?.arguments === 'string' ? e.data.arguments.length : 0,
    })
  }
  return out.reverse()
}

export function apply(ctx: Context, config: ToolTurboConfig = DEFAULT_CONFIG): void {
  if (!config.enabled) return

  // 边界适配: npm 包未 re-export 官方事件类型增强, 这里放宽签名 (第 3 章常见坑)
  const on = ctx.on as unknown as (
    event: string,
    handler: (payload: Record<string, unknown>, next: () => unknown) => unknown |
    Promise<unknown>,
  ) => void

  on('agent/request', async (payload, next) => {
    const seed = await next() as { reasoningEffort?: unknown } // ⚠️ 必须 await!
    const calls = recentToolCalls(payload.agent)
    const effort = decideEffort({
      recentCalls: calls,
      selected: config.baseline,
      allowDowngrade: config.allowDowngrade,
      allowUpgrade: config.allowUpgrade,
    })
    console.log(`[tool-turbo] calls=${JSON.stringify(calls)} =>
    reasoningEffort=${effort}`)
    return { ...seed, reasoningEffort: effort }
  })
}

```

三个关键点（都是真实踩过的坑）：

1. `next()` 是 **Promise**： `await next()` 拿到当前配置；不 `await` 直接 `spread` 会得到空对象 → **provider/model 丢失** → 报错。
2. **waterfall** 语义：监听者的返回值传给下一个监听者/最终请求。返回 `{...seed, reasoningEffort}` 就是“保留原配置 + 覆盖推理档位”。
3. `agent/request` 每步都触发： `agent-loop` 的 `buildRequest` 在每一步都会走这个 **waterfall**——所以动态决策天然按步生效。

4.5 测试

单元测试（纯函数，零依赖）：

```

import { describe, expect, it } from 'vitest'
import { decideEffort } from '../src/effort-decision.ts'

it('downgrades to low for simple tool chains', () => {

```

```
expect(decideEffort({
  recentCalls: [{ name: 'write', argsSize: 40 }],
  selected: 'high', allowDowngrade: true, allowUpgrade: true,
})).toBe('low')
})
// ... 更多分支：全新提示保持基线 / 禁用降档 / 超大载荷升 max / 混合工具升 high
```

实机验证（关键——证明“注入真的发生”）：

挂载插件（第 3 章方法）→ 重启 dsh web → 发一个创建文件的任务 → 观察 dsh 进程日志：

```
[tool-turbo] agent/request: calls=[] => reasoningEffort=high
[tool-turbo] agent/request: calls=[{"name":"write",...}] => reasoningEffort=low
```

第一轮无工具调用 → 保持基线 high；检测到 write 工具 → 下一轮降为 low。注入链路完整工作。

完整可运行代码：<https://github.com/Electricitysheep/dsh-tool-turbo>

4.6 给新手的三条开发纪律

1. 先找扩展点：要改的行为 90% 有官方钩子（agent/request、settings、conversationEvents、slots）——不要 fork 核心。
2. 逻辑抽纯函数：决策/计算逻辑与 dsh 解耦 → 单测毫秒级、覆盖全分支；实机只需验证“注入发生”。
3. 实机验证不能省：单测证明逻辑，实机日志证明接线——两个都过才算完成。

下一章：[第 5 章：实战案例](#)（规划中）—— Git 面板、HTML 草稿预览、提速插件。

第 5 章：实战案例

本章目标：用三个真实提交到开源仓库的案例，演示插件的完整开发闭环——需求 → 实现 → 测试 → 实机验证。全部是社区真实 PR，可对照源码学习。

5.1 案例一：Git 面板补全 push / pull / fetch

背景：社区插件 DSH-better-sidebar 提供 Git 面板，但只有本地操作（暂存/提交/还原），没有远端同步。

实现（4 个文件，全部走“纯函数 + 路由 + UI”分层）：

```
src/git.ts          # 纯函数层：upstreamInfo / fetchRemote / pull / push
src/index.ts        # 路由层：git.upstream / git.fetch / git.pull / git.push
src/client/GitView.tsx # UI 层：上游徽标（↓ behind ↑ ahead）+ 三个按钮
tests/git-sync.spec.ts # 集成测试：本地 bare-repo 全链路
```

关键设计（值得抄的部分）：

```
// push 只允许 --force-with-lease，绝不裸 --force —— 安全红线文档化
export async function push(cwd: string, force = false): Promise<string> {
  const args = ['push']
  if (force) args.push('--force-with-lease')
  return runGit(cwd, args)
}
```

测试（本地 bare repo，无网络、无全局 git 配置）：

```
// 覆盖：上游跟踪 / 推送 / 拉取快进 / fetch 不动 HEAD / force-with-lease 拒绝覆盖他人提交
it('force push refuses to clobber unseen remote commits', async () => {
  // 其他 clone 先推了提交；本地改写历史但不 fetch (lease 过期)
  await expect(git.push(clone, true)).rejects.toThrow()
  // 远端仍持有对方提交 —— 安全保证生效
})
```

实机验证（Playwright 操作真实 dsh web）：

- 三按钮渲染，pull/push 在无 upstream 时正确禁用
- 点击“拉取远端”→ 网络请求 `git.fetch [200]` → 面板自动刷新

PR 见：<https://github.com/omdsh-dev/DSH-better-sidebar/pull/10>

5.2 案例二：HTML 预览支持未保存草稿

背景：HTML 文件编辑后，预览只显示已保存版本（README 已知限制）。看似简单，但有安全约束：非沙箱的 `srcdoc` `iframe` 会继承父 `origin`（官方代码注释明确）。

解决：抽纯函数做“安全决策”：

```
// 沙箱开启时 dirty 草稿才用 srcdoc (opaque origin, 安全)；
// 沙箱关闭时拒绝 srcdoc，保持 route-src (跨源保证)
export function htmlPreviewTarget(input): HtmlPreviewTarget {
  if (input.isHtml && input.dirty && input.draft !== null && !input.sandboxOff) {
    return { srcDoc: input.draft }
  }
  return { src: input.routeUrl }
}
```

教训：UI 看似“加个功能”，但安全模型是硬约束——先读代码注释里的为什么，再动手。

PR 见：<https://github.com/omdsh-dev/DSH-better-sidebar/pull/11>

5.3 案例三：tool-turbo 提速插件

（第 4 章完整拆解过，此处只讲结果）

- 决策逻辑：纯函数 `decideEffort`，6/6 单测
- 注入链路：`agent/request waterfall`，实机日志证明 `high` → `low`
- 价值定位（重要修正）：简单任务 `dsh` 本来就快；提速收益在长工具链任务——每步思考降档的累计效果

5.4 三个案例的共同方法论

1. 纯函数隔离：决策/计算逻辑零依赖 → 单测全分支
2. 扩展点接入：路由/waterfall/事件——不碰核心
3. 实机验证闭环：单测（逻辑）+ 真实 `dsh` 日志/网络请求（接线）双证据

下一章：[第 6 章：进阶与性能调优](#)（规划中）。

第 6 章：进阶与性能调优

本章目标：从“能跑”到“跑得好”——推理档位策略、工具调用耗时分析、真实踩坑清单。

6.1 性能模型：dsh 的时间花在哪

实测一个“创建文件”任务的耗时分布：

阶段	占比	说明
模型思考 (Think)	~90%	每次工具调用前都会重新思考，是绝对大头
工具执行	<1%	文件写入等毫秒级
网络/渲染	~10%	API 往返 + UI 更新

推论：

- 简单任务 → 优化思考时间（降推理档）
- 长工具链任务 → 每步都省思考时间，累计收益最大
- 工具本身慢（搜索/大文件）→ 优化工具实现，不是调档

6.2 reasoning_effort 策略（官方三档）

档位	场景建议
low	简单/确定性轮次：文件操作、批量、工具链中的廉价步
high	日常 Agent 任务（默认）
max	复杂推理、长链规划、debug

手动：UI 的“推理等级”选择器，或 `~/.dsh/settings.yaml` 的 `reasoningEffort`。自动：插件按工具轮次动态降档（`dsh-tool-turbo`，第 4 章）。

6.3 工具调用耗时可视化

dsh 的会话统计行（Web UI 底部）显示：`N 轮 • M 步 | LLM Xs • 工具调用 Ys | 首 token 平均 ...`——这是最快的瓶颈定位手段。

进阶：host 插件监听工具事件做 per-tool 计时（`tool-turbo` 已内置 host 日志版本），把“哪个工具最慢”暴露出来。

6.4 常见坑清单（真实踩过，含解法）

#	坑	现象	解法
1	rc.1 依赖断裂	<code>pnpm install</code> 404 (<code>dsh-type-meta</code> 等从未发布)	依赖用 <code>^0.1.0-rc.6</code> 线
2	插件缺 main	No “exports” main defined	暴露 . 入口; “main”: “src/index.ts” 可被 <code>tsx</code> 加载
3	<code>next()</code> 忘 <code>await</code>	<code>provider/model</code> 丢失报错	<code>agent/request</code> 的 <code>next()</code> 返回 <code>Promise</code> ，必须 <code>await</code>

4	事件类型不识别	'agent/request' is not assignable to keyof Events	npm 未 re-export 类型增强, 边界放宽签名
5	client 测试跑不了	jsdom 报 window.__ModuleLoader__ undefined	client 产物依赖 dsh 引导机制, 组件测试在官方 CI 跑
6	简单任务"突然变快"的误判	1s vs 110s 差异被误归因	DeepSeek context cache 命中也会提速——A/B 测试要用全新 prompt
7	端口占用	dsh web 起不来	`netstat -ano

6.5 评测视角：官方成绩单 vs 独立实测

(结合 0813 正式版发布) 看 Agent 模型成绩单要三问：

- 1. 谁测的？官方自测（自家 Harness）vs 独立第三方（AA 等）
- 2. 什么 harness？框架不同分数差很大（官方 Terminal-Bench 87.9 vs AA 独立 79）
- 3. 验证器多严？宽松验证器（SWE-bench Verified 8.5% 假阳性）vs 严格（DeepSWE 0.3%）

dsh 的 agent/request waterfall 让模型跑分可复现——这是它相比闭源产品的工程优势。

下一章：第 7 章：生态与资源（规划中）。

第 7 章：生态与资源

本章目标：给你一份“加入 dsh 生态”的地图——官方入口、社区现状、以及参与方式。

7.1 官方入口

资源	地址	用途
官方仓库	https://github.com/deepseek-ai/deepseek-harness	源码、架构文档、issue
API 文档	https://api-docs.deepseek.com	模型、定价、API 指南
Discord	官方 README 内链接	社区讨论
Discussions	官方仓库 Discussions	提案/求助（官方当前建议的贡献入口）

注意：官方 CONTRIBUTING 明确“目前不接受外部 PR”（2026-08-13 時点）——但鼓励：

- 在 Discussions 提建议（官方会评估）
- 做 dsh-plugin 生态项目（官方点名认可的方式）
- 写教程/博客

7.2 插件生态（2026-08-13 快照）

项目	定位	状态
DSH-better-sidebar	文件管理/终端/Git/浏览器侧边栏	社区最完整插件
dsh-tool-turbo	工具调用提速（reasoning_effort 自动调节）	社区提速插件

dsh-handbook（本白皮书）	新手教程	生态文档
--------------------	------	------

发现插件：GitHub 搜 `topic:dsh-plugin` 。发布插件：给你的仓库加 `dsh-plugin topic + npm` 发布。

7.3 如何参与生态（新手路径）

- 1. 用起来： `dsh web` + 装两个社区插件，跑通日常
- 2. 小改进：给社区插件提 PR（读第 5 章的三个案例，那是完整的 PR 范式）
- 3. 发插件：从第 4 章的最小 host 插件起步，挂 `dsh-plugin topic`
- 4. 写内容：教程/测评/避坑文（官方鼓励），与本白皮书互相引用

7.4 推荐阅读路径

目标	路径
快速上手	第 2 章 → 装 better-sidebar → 日常用
开发插件	第 3 章 → 第 4 章 → 抄第 5 章案例
性能调优	第 6 章 → tool-turbo 源码
深度定制	官方 AGENTS.md（架构） → docs/architecture.md → packages/ 源码

结语

dsh 是 2026-08-13 才开源的项目——**生态的每一天都是"早期"**。白皮书会随 dsh 演进持续更新。如果某章命令失效，大概率是 rc 版本迭代所致——以官方 changelog 为准。

祝你在这个全新的生态里，抢到自己的位置。🚀

第 8 章：工具与上下文系统

本章目标：理解 dsh 的"能力引擎"——模型能调用哪些工具、上下文是怎么喂给模型的、以及长对话怎么处理。这是从"能跑"到"跑得明白"的关键一章。

8.1 官方能力包地图（60+ 包一览）

dsh 的能力全部以包形式提供（ `packages/<group>/<name>` ）。新手最需要认识的：

能力域	官方包	作用
工具 (tools)	fs/tool-fs、fs/tool-fs-search、fs/tool-str-replace-editor、shell/tool-bash、web、skill、todo	文件/终端/网页/技能/待办等可调用工具
上下文 (context)	context/*、compaction/*	请求上下文组装、长对话压缩
会话 (session)	session/*	持久化、标题、遥测
子代理 (subagent)	subagent/*	委派子任务
MCP	mcp/*	MCP 客户端（外部工具服务器）

工作流 (workflow)	workflow/*	多步工作流编排
安全 (safety)	sandbox/*、guard/*、interaction/*	沙箱、循环卫生、权限/审批
模型 (llm)	llm/*、llm-deepseek、llm-retry	模型接入、重试
技能 (skill)	skill/*	技能提供者注册表
界面 (client)	client/* (ui-conversation、ui-tool...)	Web UI 各部件

完整清单见官方仓库 `packages/AGENTS.md`。

8.2 内置工具（实测观察）

模型实际可调用的工具名是简短动词（实测记录，来自 dsh web 会话与 agent/request 日志）：

工具名	作用	备注
read	读文件	fs 能力
write	写文件	fs 能力
grep	内容搜索	fs-search
glob	文件模式匹配	fs-search
edit / str_replace_editor	精准编辑	工具结果含 locations（用于产物追踪）
bash / pwsh	执行命令	沙箱隔离
todo	待办管理	长任务规划
skill	技能调用	skill-catalog 注入

工具结果与产物追踪（重要概念）：工具的返回里带有 `locations`（文件路径），dsh 用它们做“产物文件行”（对话结尾的产物 chips 就是从这里来的）——模型改了什么文件，UI 能直接看到并可打开。

8.3 上下文是怎么喂给模型的

一次模型请求的上下文 = 系统提示 + 技能目录 + 对话历史 + 工具结果。实测在会话日志中可见：

上下文注入 @deepseek-ai/dsh-system-prompt ← 官方系统提示

上下文注入 skill-catalog ← 技能目录

dsh 的上下文机制：

- 系统提示分层：官方插件通过 `systemPrompt.section()` 注册提示片段（如 `ui-deliverables` 注册“产物文件引用”指导）
- 技能目录注入：可用技能列表进入上下文，模型按需调用
- 工具 schema：每步请求携带工具定义

8.4 长对话：compaction（压缩）

长对话会撑爆上下文。dsh 的 `compaction` 插件（如 `compaction-basic`）负责：

- 检测上下文溢出 (`agent/request-error` 的 `CONTEXT_WINDOW_EXCEEDED`)
- 压缩历史 (模型无关的修剪 + 可选摘要)
- 失败时路由到“溢出代理” (overflow agent)

对新手：知道“长对话会自动压缩”即可，细节是进阶话题。生产环境注意：压缩会丢细节，重要上下文建议手动写进提示词。

8.5 权限与安全模型（了解即可）

- 访问模式：UI 里可见「Workspace Write」等模式（权限预设）
- 交互审批： `interaction/*` 提供权限/审批能力（危险操作可要求确认）
- 沙箱： `sandbox/*` 隔离命令执行（如 pwsh 沙箱有 ACL 约束——实测中遇到过 temp 目录权限问题）
- 工具超时： `guard/*` 提供 loop 卫生与工具超时

安全配置的深度话题超出本白皮书范围；核心认知：dsh 的工具执行默认有隔离与审批层，不是裸执行。

8.6 新手最该记住的三件事

1. 工具名是简短动词 (read/write/grep/glob/edit/bash) ——写提示词/插件时直接说“读文件”“搜索”即可
2. 工具返回 locations → 产物追踪——模型改的文件会出现在对话产物区
3. 长对话自动压缩——不必手动清理历史（但重要信息要写进提示词）

下一章： [第 9 章：MCP、子代理与 workflow](#)

第 9 章：MCP、子代理与 workflow

本章目标：了解 dsh 的三项扩展能力——MCP（接入外部工具）、子代理（并行任务）、workflow（多步编排）。这些是把 dsh 从“单 Agent”升级为“Agent 系统”的开关。

⚠ 本章基于官方架构文档 (`packages/AGENTS.md`) 与包结构整理；部分功能示例标注「待实测」——rc 版本迭代快，具体用法以官方 changelog 为准。

9.1 MCP：接入外部工具生态

MCP (Model Context Protocol) 是“给 Agent 插外部工具”的开放协议。dsh 提供 `mcp` 客户端包 (`@deepseek-ai/dsh-mcp-client`)。

能做什么：通过 MCP 接入任何 MCP 服务器（数据库、浏览器、图表、公司内部系统……）——比如社区已出现的 `dsh-plugin-cost-tracker`（追踪 token）就是 MCP/插件形态。

新手定位：MCP 是“生态扩展”，等你在 dsh 里有了明确的工具缺口再来接。先跑通内置工具，再加 MCP。

9.2 子代理 (subagent)：并行干活

是什么：把任务委派给子 Agent 并行执行 (`packages/subagent/*`)。

典型场景：

- 大仓库多模块调研（各模块一个子代理）
- 长任务分解（父代理规划，子代理执行）
- 独立验证（子代理交叉检查）

对新手：子代理是“高阶武器”——先用好单 Agent，理解任务分解后再上子代理。错误示范是“什么任务都开子代理”，反而增加协调开销。

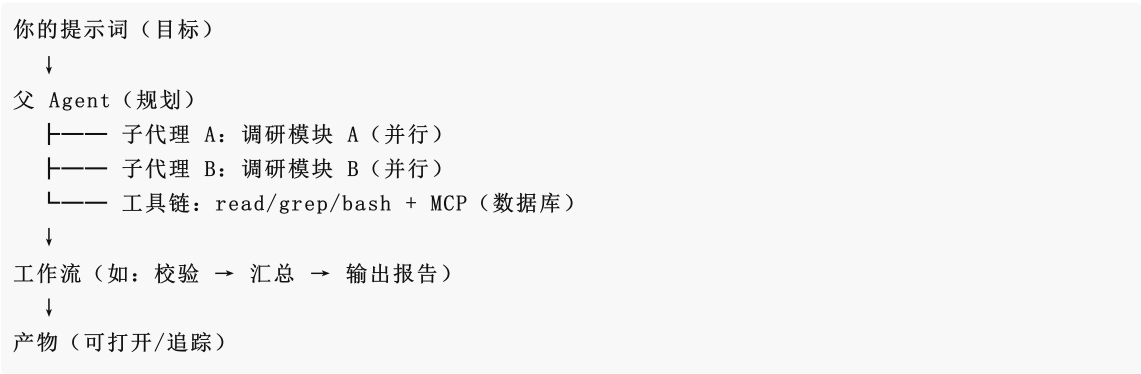
9.3 工作流 (workflow)：多步编排

是什么： `packages/workflow/*` 提供多步工作流编排 (worker-thread provider + tool Consumer)。

和"多轮对话"的区别：普通对话是"模型自由发挥"，工作流是"把步骤定义成流程，按顺序/条件执行"——适合确定性流程（如：拉取数据 → 清洗 → 生成报表 → 校验）。

官方示例（ `packages/examples/` ）：有可运行的 `cordis.yml` 工作流示例。

9.4 组合起来：一个"Agent 系统"长什么样



新手路径建议：

- 1. 阶段一：单 Agent + 内置工具（第 2-5 章）
- 2. 阶段二：+ MCP（接外部工具）
- 3. 阶段三：+ 子代理（并行）
- 4. 阶段四：+ 工作流（确定性流程）

9.5 社区生态示例（2026-08-13 快照）

官方 Discussion 里已出现的社区项目：

- `dsh-plugin-cost-tracker` ——实时追踪 token 成本（插件/MCP 形态）
- 插件开发/提速类（如本白皮书配套的 `dsh-tool-turbo`）

生态正在快速生长——现在是进场搭积木的好时候。

附录 A：[术语表与命令速查](#)

附录 A：术语表与命令速查

术语表

术语	一句话解释
Harness	套在模型外的工程层：会话、工具、上下文、循环控制
dsh	DeepSeek Harness 的命令行名（dsh 命令）
Profile	一种可启动形态（web / headless / 自定义），= bundle 栈 + patch 层
Bundle	一组插件的集合（官方：dsh-base、dsh-web-app、dsh-headless）

Cordis	dsh 底层的插件容器（依赖注入、事件、生命周期）
插件 (Plugin)	cordis 插件；可同时携带 host 半 (Node) 与 client 半 (浏览器)
host 半 / client 半	插件在 Node 侧 (工具/服务/事件) 与浏览器侧 (UI) 的两副面孔
扩展点	官方提供的钩子 (agent/request、settings、conversationEvents、slots)
agent/request waterfall	每步模型请求前可改配置的事件链（插件在此注入 reasoningEffort 等）
reasoning_effort	思考强度 (low/high/max) ——速度与质量的权衡旋钮
headless	一次性 CLI 任务模式 (dsh --profile headless “任务”)
compaction	长对话上下文压缩
subagent	子代理（并行委派任务）
MCP	模型上下文协议（接入外部工具服务器）
workflow	多步确定性工作流编排
locations	工具返回的文件路径（驱动产物追踪）
产物文件	模型创建/修改的文件（对话末尾的可打开 chips）
sandbox	命令执行的隔离沙箱
reasoning_effort	同「思考强度」
rc	预发布版本 (0.1.0-rc.6) ，迭代快、可能有破坏性变更

命令速查

dsh 核心

```
dsh web                                # 启动 Web UI
dsh --profile headless “任务”         # 一次性任务（脚本/CI）
dsh --version                          # 版本
dsh --dump-config                      # 合成配置树
dsh plugin --profile <name> add <pkg> # 安装插件
```

环境

```
node --version                        # 需 ≥ 22
npm install -g @deepseek-ai/dsh      # 全局安装
npx -y @deepseek-ai/dsh web          # 免安装运行
```

排障

```
netstat -ano | findstr 3080           # 端口占用 (Windows)
taskkill /PID <pid> /F                 # 杀进程
cat ~/.dsh/settings.yaml              # 全局配置（模型/推理档位）
```

插件开发

```
# 挂载到 profile (web 为例)
# ① package.json 加依赖: "pkg": "link:C:\\path\\to\\pkg"
# ② cordis.patch.yml 加:
#   - insert:
#     - id: <插件id>
#       name: <npm包名>
cd ~/.dsh/profiles/web && pnpm install          # 安装
```

配置参考 (settings.yaml)

```
agent-default-model:
  model: deepseek-v4-flash      # 或 deepseek-v4-pro
  reasoningEffort: high        # low / high / max
```

附录 B: [同模型多 Agent 实测](#)

附录：同模型 × 不同 Agent 实测对比（Benchmark）

实测日期: 2026-08-13 | 模型: deepseek-v4-flash (同一 opencode-go 网关, 同一 API key) | Agent: dsh / opencode / omp 目的: 控制模型变量, 对比 Agent 工程层的差异 (提示词、工具链、轮次管理)。

方法

项	说明
模型	deepseek-v4-flash (三 agent 均走 opencode-go 网关 + 同一 key, 公平对照)
Agent	dsh (headless profile)、opencode (opencode run)、omp (--print 非交互)
任务	T1 创建文件 / T2 搜索统计 / T3 修复 bug + 验证
环境	Windows 11 + Node 24, 独立工作目录, 无预置上下文
度量	耗时 (秒) + 正确性 (人工核对产物/输出)

结果 (3 轮采样, 取中位数)

Agent	T1 创建文件	T2 搜索统计	T3 修 bug+验证	总计 (中位)	正确率
omp	10s	11s	15s	36s	27/27
dsh	22s	15s	48s	85s	27/27
opencode	35s	37s	42s	114s	27/27

原始 3 轮 (秒)：

Agent	轮次	T1	T2	T3
-------	----	----	----	----

dsh	1 / 2 / 3	11 / 27 / 22	11 / 15 / 22	27 / 48 / 74
opencode	1 / 2 / 3	18 / 35 / 35	19 / 37 / 37	50 / 41 / 42
omp	1 / 2 / 3	9 / 10 / 12	9 / 11 / 12	13 / 33 / 15

解读

- 1. 正确率三家持平 (27/27) ——同模型下，Agent 工程层的差异主要体现在效率而非能力上限（对这三个任务而言）。
- 2. 总耗时稳定排序：omp < dsh < opencode (3 轮一致)。
- 3. 任务类型影响排序：T3（多步：读→改→运行验证）波动最大（dsh 27→74s），说明复杂任务对 Agent 的轮次管理/工具效率更敏感。
- 4. dsh 定位：dsh 居中，且 headless 是"运行时 + 插件生态"——同样的模型，通过插件（如 tool-turbo 降推理档）可进一步优化耗时（见第 6 章）。
- 5. 样本警示：3 轮 × 3 任务，同网关同 key，含网络抖动——方向可信，绝对值会随环境波动。

可复现

```
# 三个 agent 的命令（独立目录）
dsh --profile headless "任务"
opencode run "任务" --model opencode-go/deepseek-v4-flash
omp "任务" --model deepseek-v4-flash --print
```

完整任务定义与产物见本仓库 [benchmark/](#) 目录 (规划中)。